

TUGAS PRAKTIKUM 5

Struktur Data



RAFKI AHMAD PAGAMANDA

2311533016

Kelas D

Implementasi Single Linked List

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

Implementasi Single Linked List

A. Tugas

1. Program Pasien

```
1 package pekan5_2311533016;
2
3 public class Pasien_2311533016 {
4
5     private String namaPasien_3016;
6     private String keluhan_3016;
7     private int nomorAntrian_3016;
8
9     Pasien_2311533016 next_3016;
10
11     // constructor
12     public Pasien_2311533016(String namaPasien_3016,
13         String keluhan_3016,
14         int nomorAntrian_3016) {
15
16         this.namaPasien_3016 = namaPasien_3016;
17         this.keluhan_3016 = keluhan_3016;
18         this.nomorAntrian_3016 = nomorAntrian_3016;
19         this.next_3016 = null;
20     }
21
22     // getter
23     public String getNamaPasien_3016() {
24         return namaPasien_3016;
25     }
26
27     public String getKeluhan_3016() {
28         return keluhan_3016;
29     }
30
31     public int getNomorAntrian_3016() {
32         return nomorAntrian_3016;
33     }
34
35     public Pasien_2311533016 getNext_3016() {
36         return next_3016;
37     }
38
39     // setter
40     public void setNamaPasien_3016(String namaPasien_3016) {
41         this.namaPasien_3016 = namaPasien_3016;
42     }
43
44     public void setKeluhan_3016(String keluhan_3016) {
45         this.keluhan_3016 = keluhan_3016;
46     }
47
48     public void setNomorAntrian_3016(int nomorAntrian_3016) {
49         this.nomorAntrian_3016 = nomorAntrian_3016;
50     }
51 }
```

Pada class Pasien_2311533016, sy membuat sebuah ADT (Abstract Data Type) yang digunakan untuk menyimpan data pasien pada sistem antrian rumah sakit. Class ini berfungsi sebagai node pada Single Linked List, dimana setiap node menyimpan informasi pasien serta pointer yang menunjuk ke node berikutnya.

Di dalam class terdapat beberapa atribut yaitu namaPasien_3016, keluhan_3016, dan nomorAntrian_3016. Variabel tersebut digunakan untuk menyimpan nama pasien, jenis keluhan atau penyakit pasien, serta nomor antrian pasien. Selain itu terdapat pointer next_3016 yang berfungsi untuk menghubungkan node saat ini dengan node pasien berikutnya pada linked list.

Pada constructor, sy menginisialisasi seluruh atribut ketika object pasien dibuat. Data nama pasien, keluhan, dan nomor antrian langsung dimasukkan melalui parameter constructor sehingga object dapat langsung digunakan. Pointer next_3016 diisi null sebagai tanda bahwa node belum terhubung ke node lain.

Class ini juga memiliki method getter dan setter. Getter digunakan untuk mengambil data dari atribut, sedangkan setter digunakan untuk mengubah nilai atribut maupun pointer next_3016. Dengan adanya getter dan setter, pengolahan data pasien menjadi lebih mudah dan terstruktur.

Output dari class ini sendiri tidak langsung terlihat karena class hanya digunakan sebagai pembentuk object pasien dan node linked list. Namun class ini menjadi dasar utama agar data pasien dapat diproses pada program antrian rumah sakit.

2. Rumah Sakit

```
1 package pekan5_2311533016;
2
3 import java.util.Scanner;
4
5 public class RumahSakit_2311533016 {
6
7     static Pasien_2311533016 head_3016 = null;
8     static int counter_3016 = 0;
9
10    // insert tail
11    public static void daftarPasien_3016(String nama_3016, String keluhan_3016) {
12
13        counter_3016++;
14
15        Pasien_2311533016 pasienBaru_3016 =
16            new Pasien_2311533016(
17                nama_3016,
18                keluhan_3016,
19                counter_3016);
20
21        // jika list kosong
22        if (head_3016 == null) {
23            head_3016 = pasienBaru_3016;
24        } else {
25
26            Pasien_2311533016 temp_3016 = head_3016;
27
28            while (temp_3016.next_3016 != null) {
29                temp_3016 = temp_3016.next_3016;
30            }
31
32            temp_3016.next_3016 = pasienBaru_3016;
33        }
34
35        System.out.println(
36            "Pasien berhasil didaftarkan! Nomor Antrian: "
37            + counter_3016);
38    }
39
40    // delete head
41    public static void panggilPasien_3016() {
42
43        if (head_3016 == null) {
44            System.out.println("Antrian kosong!");
45            return;
46        }
47
48        System.out.println("Pasien dipanggil:");
49        System.out.println("Nama : "
50            + head_3016.getNamaPasien_3016());
```

```

51         System.out.println("Keluhan : "
52             + head_3016.getKeluhan_3016());
53
54         System.out.println("Nomor Antrian : "
55             + head_3016.getNomorAntrian_3016());
56
57         head_3016 = head_3016.next_3016;
58     }
59
60     // display
61     public static void tampilkanAntrian_3016() {
62
63         if (head_3016 == null) {
64             System.out.println("Antrian kosong!");
65             return;
66         }
67
68         Pasien_2311533016 temp_3016 = head_3016;
69
70         System.out.println("=== DAFTAR ANTRIAN ===");
71
72         while (temp_3016 != null) {
73
74             System.out.println(
75                 "No: "
76                 + temp_3016.getNomorAntrian_3016());
77
78             System.out.println(
79                 "Nama: "
80                 + temp_3016.getNamaPasien_3016());
81
82             System.out.println(
83                 "Keluhan: "
84                 + temp_3016.getKeluhan_3016());
85
86             System.out.println("-----");
87
88             temp_3016 = temp_3016.next_3016;
89         }
90     }
91
92     // search
93     public static void cariPasien_3016(String cari_3016) {
94
95         Pasien_2311533016 temp_3016 = head_3016;
96
97         boolean ditemukan_3016 = false;
98
99         while (temp_3016 != null) {
100
101             if (temp_3016.getNamaPasien_3016()
102                 .equalsIgnoreCase(cari_3016)) {
103
104                 System.out.println("Pasien ditemukan!");
105                 System.out.println(
106                     "Nomor Antrian: "
107                     + temp_3016.getNomorAntrian_3016());
108
109                 System.out.println(
110                     "Keluhan: "
111                     + temp_3016.getKeluhan_3016());
112
113                 ditemukan_3016 = true;
114                 break;
115             }
116
117             temp_3016 = temp_3016.next_3016;
118         }
119
120         if (!ditemukan_3016) {
121             System.out.println("Pasien tidak ditemukan!");
122         }
123     }
124
125     // status antrian
126     public static void cekStatusAntrian_3016() {
127
128         if (head_3016 == null) {
129             System.out.println("Antrian kosong!");
130             return;
131         }
132
133         int jumlah_3016 = 0;
134
135         Pasien_2311533016 temp_3016 = head_3016;
136
137         while (temp_3016 != null) {
138             jumlah_3016++;
139             temp_3016 = temp_3016.next_3016;
140         }
141
142         System.out.println("Jumlah pasien: " + jumlah_3016);
143
144         System.out.println(
145             "Pasien terdepan: "
146             + head_3016.getNamaPasien_3016());
147     }
148
149     // main
150

```

```

151 public static void main(String[] args) {
152
153     Scanner input_3016 = new Scanner(System.in);
154
155     int pilihan_3016;
156
157     do {
158
159         System.out.println(
160             "\n=== Antrian Rumah Sakit 2311533016 ===");
161
162         System.out.println("1. Daftarkan Pasien");
163         System.out.println("2. Panggil Pasien");
164         System.out.println("3. Tampilkan Antrian");
165         System.out.println("4. Cari Pasien");
166         System.out.println("5. Cek Status Antrian");
167         System.out.println("6. Keluar");
168
169         System.out.print("Pilihan: ");
170         pilihan_3016 = input_3016.nextInt();
171         input_3016.nextLine();
172
173         switch (pilihan_3016) {
174
175             case 1:
176
177                 System.out.print("Nama Pasien: ");
178                 String nama_3016 =
179                     input_3016.nextLine();
180
181                 System.out.print("Keluhan: ");
182                 String keluhan_3016 =
183                     input_3016.nextLine();
184
185                 daftarPasien_3016(
186                     nama_3016,
187                     keluhan_3016);
188
189                 break;
190
191             case 2:
192
193                 panggilPasien_3016();
194                 break;
195
196             case 3:
197
198                 tampilkanAntrian_3016();
199                 break;
200
201             case 4:
202
203                 System.out.print(
204                     "Masukkan nama pasien: ");
205
206                 String cari_3016 =
207                     input_3016.nextLine();
208
209                 cariPasien_3016(cari_3016);
210
211                 break;
212
213             case 5:
214
215                 cekStatusAntrian_3016();
216                 break;
217
218             case 6:
219
220                 System.out.println("Program selesai");
221                 break;
222
223             default:
224
225                 System.out.println("Pilihan tidak valid");
226
227         }
228     } while (pilihan_3016 != 6);
229
230     input_3016.close();
231 }
232

```

Pada class RumahSakit_2311533016, sy membuat program simulasi antrian rumah sakit menggunakan konsep Single Linked List. Program ini bekerja menggunakan prinsip FIFO (First In First Out), dimana pasien yang pertama mendaftar akan menjadi pasien pertama yang dipanggil.

Program memiliki variabel head_3016 yang digunakan sebagai penunjuk node pertama pada linked list, serta counter_3016 yang digunakan untuk memberikan nomor antrian secara otomatis kepada setiap pasien baru. Ketika pasien didaftarkan, nomor antrian akan bertambah secara otomatis sesuai urutan pendaftaran.

Method `daftarPasien_3016()` digunakan untuk menambahkan pasien baru ke bagian akhir linked list (insert tail). Jika linked list masih kosong, maka node pasien baru langsung menjadi head. Namun jika sudah terdapat data, program akan menelusuri node hingga bagian terakhir lalu menghubungkannya dengan node baru. Output dari method ini adalah munculnya pesan bahwa pasien berhasil didaftarkan beserta nomor antrian pasien.

Method `panggilPasien_3016()` digunakan untuk memanggil pasien terdepan sekaligus menghapus node paling depan pada linked list (delete head). Setelah pasien dipanggil, posisi head akan dipindahkan ke node berikutnya sehingga antrian terus berjalan sesuai urutan. Output dari method ini adalah data pasien yang dipanggil, seperti nama pasien, keluhan, dan nomor antrian.

Method `tampilkanAntrian_3016()` digunakan untuk menampilkan seluruh data pasien yang ada pada antrian. Program akan menelusuri linked list mulai dari head hingga node terakhir, kemudian menampilkan nomor antrian, nama pasien, dan keluhan pasien. Output yang dihasilkan berupa daftar seluruh pasien yang masih berada dalam antrian.

Method `cariPasien_3016()` digunakan untuk mencari data pasien berdasarkan nama. Proses pencarian dilakukan menggunakan `equalsIgnoreCase()` sehingga pencarian tidak membedakan huruf besar dan kecil. Jika data ditemukan maka informasi pasien akan ditampilkan, sedangkan jika tidak ditemukan program akan menampilkan pesan bahwa pasien tidak ada. Output method ini berupa status apakah pasien ditemukan atau tidak beserta data pasien tersebut.

Method `cekStatusAntrian_3016()` digunakan untuk melihat kondisi antrian saat ini, seperti jumlah total pasien dan informasi pasien yang berada di posisi paling depan. Jika linked list kosong maka program akan menampilkan pesan bahwa antrian kosong. Output dari method ini adalah informasi jumlah pasien dan pasien yang sedang berada di urutan terdepan.

Pada method `main`, sy menggunakan Scanner untuk menerima input dari user. Program dibuat dalam bentuk menu interaktif sehingga user dapat memilih operasi yang ingin dijalankan, seperti menambah pasien, memanggil pasien, menampilkan antrian, mencari pasien, hingga keluar dari program. Output utama program adalah tampilan menu interaktif yang akan terus muncul sampai user memilih menu keluar.

Hasil akhir dari program ini menunjukkan bahwa konsep Single Linked List dapat digunakan untuk membuat simulasi antrian rumah sakit secara dinamis. Program mampu menambahkan, menghapus, mencari, dan menampilkan data pasien dengan baik sesuai prinsip FIFO.

B. Kesimpulan

Berdasarkan tugas praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Single Linked List dapat digunakan untuk membuat sistem antrian rumah sakit secara dinamis dengan menerapkan prinsip FIFO (First In First Out). Setiap pasien direpresentasikan sebagai node yang saling terhubung menggunakan pointer next, sehingga proses penambahan dan penghapusan data dapat dilakukan dengan lbh mudah.

Pada program yang dibuat, operasi insert digunakan untuk menambahkan pasien ke bagian akhir antrian, sedangkan delete head digunakan untuk memanggil pasien yang berada di urutan paling depan. Selain itu, program juga mampu menampilkan seluruh data pasien, mencari pasien berdasarkan nama, serta mengecek kondisi antrian saat ini.

Hasil dari program menunjukkan bahwa konsep Single Linked List dapat diterapkan dengan baik dalam simulasi antrian rumah sakit. Program mampu menjalankan seluruh operasi sesuai instruksi tugas tanpa error serta dapat membantu memahami cara kerja pointer dan hubungan antar node dalam linked list.